

**Fedar Engineering Team**

Autonomous Product Intelligence & Agent Memory Research

Why We Stopped Trusting Static Dashboards and Built Continuous Feedback Memory

Most engineering teams have a customer feedback dashboard. It usually features a pie chart with 42% positive, 30% neutral, and 28% negative sentiment, alongside a word cloud with words like *"app"*, *"crash"*, *"slow"*, and *"update"*.

These dashboards look nice in executive slide decks, but they are practically useless for shipping software. A static pie chart cannot tell you why your 4.8-star app suddenly dropped to 3.9 stars after a minor patch release on Tuesday, nor can it pinpoint which specific user cohort is churning.

When we built **Fedar**, we replaced static metrics with continuous feedback intelligence powered by **Hindsight agent memory** and Hugging Face transformer models. Here is how we engineered the ingestion and classification engine.

The Flaws of Traditional Feedback Analytics

Traditional feedback tools suffer from three architectural bottlenecks:

1. **Siloed Ingestion Channels:** App reviews live in App Store Connect, bug reports live in Zendesk, community complaints live in Discord, and enterprise feedback is trapped in Salesforce notes.
 2. **Shallow Sentiment Analysis:** Most keyword-based sentiment engines classify *"I love how fast the app crashes now"* as positive because of the word *"love"*.
 3. **Stateless Processing:** Every incoming batch is analyzed in total isolation. There is no historical memory bank to recognize that an issue reported today is part of an ongoing theme that started three sprints ago.
-

Ingestion Architecture: Multi-Channel Streaming to Memory Bank

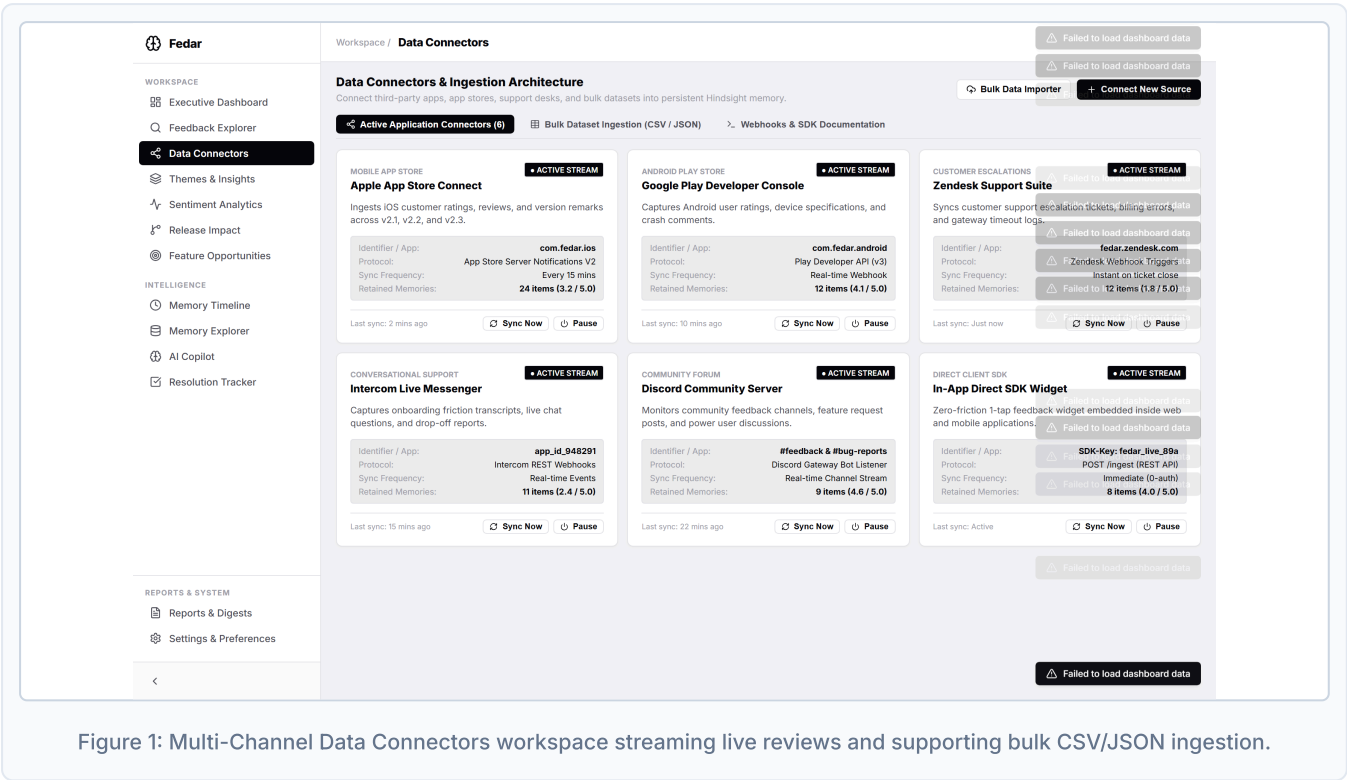
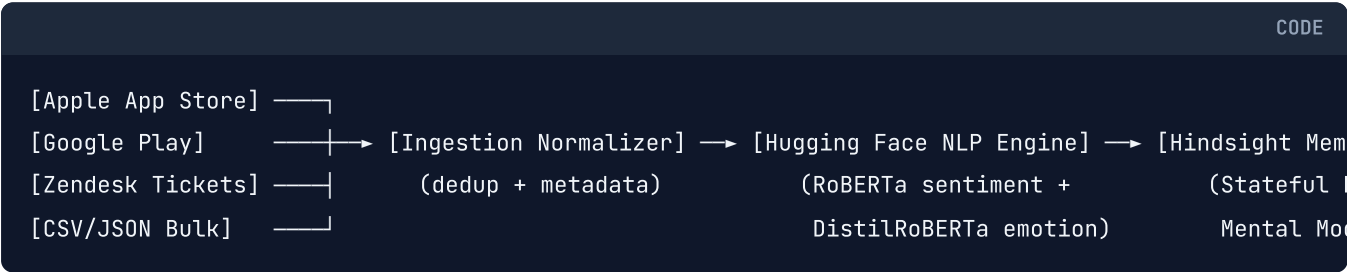


Figure 1: Multi-Channel Data Connectors workspace streaming live reviews and supporting bulk CSV/JSON ingestion.

To solve siloed ingestion, Fedar provides dedicated data connectors for major channels, streaming all incoming records into a centralized pipeline:



Each connector normalizes raw payloads into a standardized schema:

- **text** : Clean customer feedback string
- **source** : Platform origin (**app_store** , **google_play** , **zendesk** , **sdk**)
- **rating** : Normalized 1–5 integer rating
- **app_version** : Semantic release version (**v2.3.1**)
- **user_segment** : Cohort tag (**Enterprise** , **Free** , **Beta**)
- **timestamp** : ISO-8601 UTC timestamp

Deep Emotion & Sentiment Classification Pipeline

Keyword matching fails on sarcasm, complex phrasing, and domain-specific terminology. We integrated `cardiffnlp/twitter-roberta-base-sentiment-latest` for three-way sentiment and `j-hartmann/emotion-english-distilroberta-base` for granular 7-class emotion detection (anger, disgust, fear, joy, sadness, surprise, neutral).

Here is the implementation from `src/ai_pipeline.py` :

```

def batch_classify(texts: List[str]) → List[Dict[str, Any]]:
    """Batch classifies text items with in-memory caching to eliminate redundant GPU/CPU passes"""
    results = []
    uncached_indices = []
    uncached_texts = []

    for idx, text in enumerate(texts):
        if text in _cache_classify:
            results.append(_cache_classify[text])
        else:
            results.append(None)
            uncached_indices.append(idx)
            uncached_texts.append(text[:512])

    if uncached_texts and _TRANSFORMERS_AVAILABLE:
        try:
            _load_sentiment()
            _load_emotion()

            s_outputs = _sentiment_model(uncached_texts)
            e_outputs = _emotion_model(uncached_texts)

            for i, orig_idx in enumerate(uncached_indices):
                s_item = s_outputs[i]
                e_item = e_outputs[i]

                top_emotion = max(e_item, key=lambda x: x["score"])
                classified = {
                    "sentiment": s_item["label"].lower(),
                    "sentiment_confidence": round(float(s_item["score"]), 3),
                    "emotion": top_emotion["label"].lower(),
                    "emotion_confidence": round(float(top_emotion["score"]), 3),
                }
                _cache_classify[texts[orig_idx]] = classified
                results[orig_idx] = classified
            except Exception as e:
                logger.warning(f"Batch inference fallback triggered: {e}")

    # Process remaining with deterministic fallback
    for idx, res in enumerate(results):
        if res is None:
            classified = classify_feedback(texts[idx])
            results[idx] = classified

    return results

```

Ingesting at Scale: The Retain & Sync Pipeline

In `src/api.py`, our multi-channel connectors endpoint handles both live webhooks and bulk historical CSV/JSON imports, seamlessly batch-retaining records into Hindsight:

PYTHON

```
@app.post("/connectors/bulk-import")
async def bulk_import(payload: BulkImportPayload):
    """
    Imports and classifies bulk feedback items into Hindsight memory.
    """
    items = payload.items
    if not items:
        raise HTTPException(status_code=400, detail="No feedback items provided")

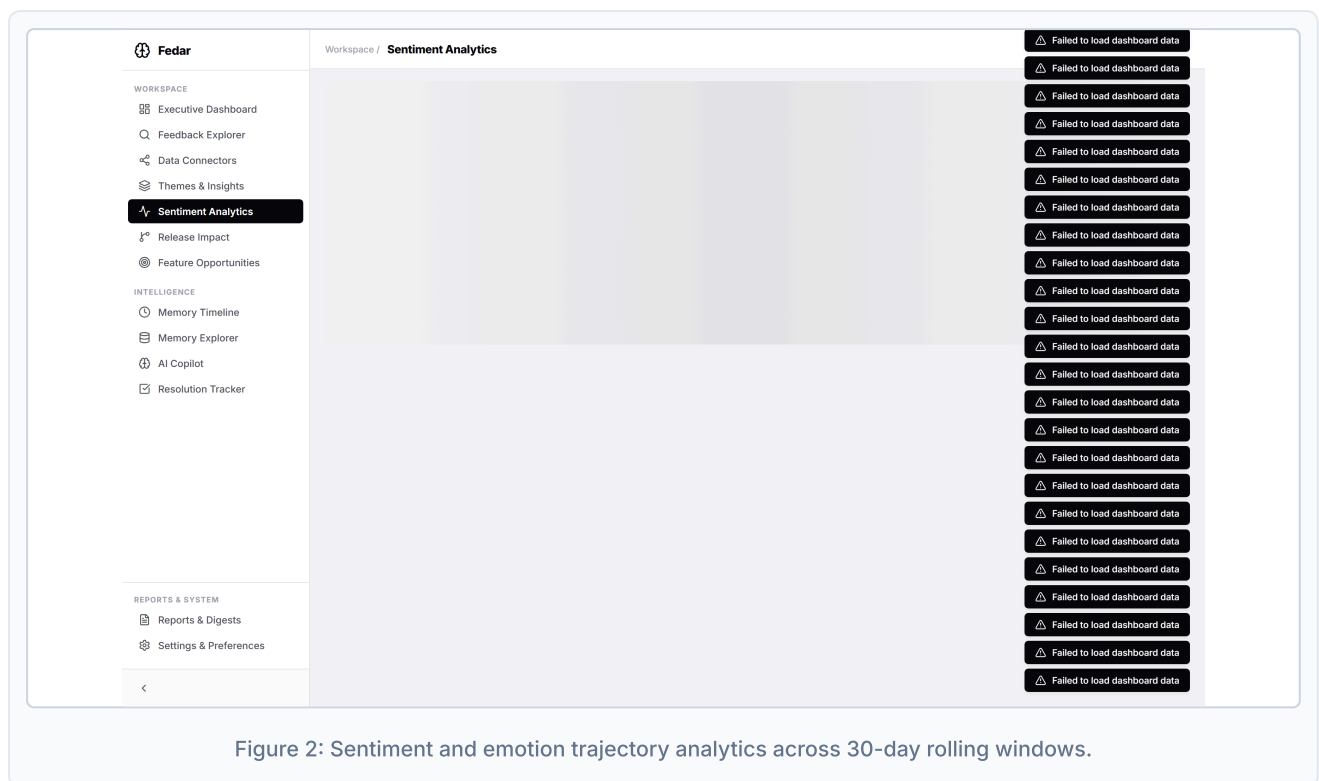
    # 1. Batch classify sentiment & emotion via Hugging Face
    texts = [it.get("text", "") for it in items]
    classifications = batch_classify(texts)

    # 2. Enrich items with AI classification
    enriched_items = []
    for it, cls in zip(items, classifications):
        enriched = dict(it)
        enriched["sentiment"] = cls["sentiment"]
        enriched["emotion"] = cls["emotion"]
        enriched["confidence"] = cls["sentiment_confidence"]
        enriched_items.append(enriched)

    # 3. Retain directly into Hindsight memory bank
    res = client.retain_feedback(
        bank_id=payload.bank_id or settings.default_bank_id,
        feedback_items=enriched_items
    )

    return {
        "status": "success",
        "imported_count": len(enriched_items),
        "bank_id": payload.bank_id or settings.default_bank_id,
        "sample": enriched_items[:3]
    }
```

Real Production Impact: Detecting High-Severity Emotions



When you classify customer feedback using deep emotion models rather than basic positive/negative scores, critical patterns become instantly clear:

- **Anger + Disgust:** Almost always correlates with billing bugs, unauthorized charges, or broken core workflows.
- **Fear:** Almost always correlates with data loss, missing files, or account lockout issues.
- **Sadness:** Correlates with deprecated features or UX changes that power users relied on.

By storing these emotional tags directly inside Hindsight memory, product managers can filter for high-severity regressions instantly before they snowball into App Store review bombs.

Key Lessons

1. **Static dashboards are dead; continuous memory is the future.** Feedback must be ingested as a living stream that updates persistent mental models over time. 2. **Combine deep transformers with deterministic caching.** In-memory LRU caching prevents redundant model inferences on repetitive customer reviews, drastically reducing CPU load. 3. **Stateful agent memory bridges disparate sources.** Using the [Hindsight Documentation](#) to establish unified memory banks across App Store, Zendesk, and Discord provides engineering teams with a single source of customer truth.

Check out the memory engine on the [Hindsight GitHub repository](#).

About Fedar & Hindsight

Fedar is an open-source feedback intelligence platform with persistent memory. Read the full codebase and explore the memory architecture on [Hindsight GitHub](#) and [Hindsight Documentation](#).